

# Practico-G3-Segundo-Parcial.pdf



andreagb03



Inteligencia Artificial



3º Grado en Ingeniería Informática - Tecnologías Informáticas



Escuela Técnica Superior de Ingeniería Informática  
Universidad de Sevilla



[Accede al documento original](#)

**CURSO NUEVO EQUIPO NUEVO**

**CeX**  
webuy.com



27/12/24, 12:20

Práctico G3 Segundo Parcial

## Inteligencia Artificial

### Parte práctica de la segunda prueba de evaluación alternativa.

```
In [1]: # Apellidos:  
# Nombre:
```

Cuando un conjunto de entrenamiento tiene valores numéricos, no podemos aplicar directamente el algoritmo ID3 para construir un árbol de decisión. Para poder aplicar ese algoritmo, debemos transformar el conjunto de entrenamiento en otro con valores discretos. En los siguientes ejercicios vamos a implementar algunas funciones relacionadas con dicha transformación. Representaremos un conjunto de entrenamiento como un diccionario donde una de las claves tiene por valor 'Atributos' y representa los atributos y en el resto de los pares clave-valor, las claves son tuplas y el valor será 1 o 0. Nótese que con este sistema de representación con diccionarios, no puede haber ejemplos duplicados en un conjunto de entrenamiento. Ejemplos de conjuntos de entrenamiento:

```
In [7]: D1 = {( 'Cielo', 'Temperatura', 'Humedad', 'Viento'): 'Atributos',  
             ( 'Soleado', 17, 'NO', 12.1):0,  
             ( 'Soleado', 26, 'NO', 12.5):0,  
             ( 'Nublado', 18, 'NO', 12.3):0,  
             ( 'Lluvia', 24, 'NO', 15.1):1,  
             ( 'Lluvia', 23, 'SI', 14.8):1,  
             ( 'Nublado', 27, 'SI', 12.2):0,  
             ( 'Soleado', 30, 'NO', 16.4):1,  
             ( 'Soleado', 22, 'SI', 13.7):1,  
             ( 'Lluvia', 21, 'SI', 16.9):1,  
             ( 'Soleado', 29, 'SI', 13.6):1,  
             ( 'Nublado', 20, 'NO', 17.3):1,  
             ( 'Nublado', 28, 'SI', 13.1):1,  
             ( 'Lluvia', 19, 'NO', 13.0):0}  
  
D2 = {(1.2, 'B1', 3):1,  
      (3.1, 'B1', 7):0,  
      (1.4, 'B2', 4):1,  
      (1.5, 'B3', 5):1,  
      ('AT1', 'AT2', 'AT3'): 'Atributos',  
      (1.4, 'B2', 2):1,  
      (2.0, 'B3', 6):0,  
      (2.3, 'B3', 10):0,  
      (3.6, 'B1', 13):0,  
      (4.0, 'B3', 1):1}  
  
D3 = {(1, 'T1'):0,  
      (2, 'T2'):0,  
      (3, 'T3'):1,  
      (4, 'T4'):1,  
      ('K1', 'K2'): 'Atributos'}
```



Para simplificar los ejercicios, supondremos siempre que los ejemplos que toman valores iguales en algún atributo numérico, tienen el mismo valor de clasificación. Esto es, no vamos a encontrar dos ejemplos E0 y E1 en los que el Atributo AT tome el mismo valor V y E0 tenga clasificación 0 y E1 tenga clasificación 1.

## Ejercicio 1 [0.5 ptos]:

Escribir una función `at_num(D)` que recibe como entrada un conjunto de entrenamiento `D` y devuelve la lista de atributos que toman valores numéricos en `D`. **Nota:** Los valores numéricos serán de tipo `float` o `int`, por tanto, para comprobar si un valor `x` es de tipo numérico, puedes usar `isinstance(x,(int,float))`.

```
In [16]: # Solución
def at_num(D):
    # Obtener los nombres de los atributos
    atributos = D.get('Atributos', next((k for k, v in D.items() if v == 'Atributos'))
    # Inicializar lista de atributos numéricos
    atributos_numericos = []

    # Iterar por índices de los atributos
    for i, atributo in enumerate(atributos):
        # Verificar si el atributo toma valores numéricos
        valores_atributo = (ej[i] for ej in D if isinstance(ej, tuple) and D[ej]
        if all(isinstance(v, (int, float)) for v in valores_atributo):
            atributos_numericos.append(atributo)

    return atributos_numericos
```

Puedes probar con los siguientes ejemplos

```
In [19]: at_num(D1)
# Salida:
# ['Temperatura', 'Viento']
```

```
Out[19]: ['Temperatura', 'Viento']
```

```
In [21]: at_num(D2)
# Salida:
# ['AT1', 'AT3']
```

```
Out[21]: ['AT1', 'AT3']
```

```
In [12]: at_num(D3)
# Salida:
# ['K1']
```

```
Out[12]: ['K1']
```

## Ejercicio 2 [1 pto]:

Dado un atributo, definimos un punto de corte como el valor medio de los valores  $x_1$  y  $x_2$  del atributo, siempre que  $x_1$  y  $x_2$  sean dos valores consecutivos (una vez ordenados) y la clasificación de los ejemplos correspondientes cambien. Se pide definir la función `cortes(D,At)` que reciba un conjunto de entrenamiento  $D$  y un atributo de valores numéricos  $At$  y devuelva los puntos de corte.

```
In [23]: # Solución:
def cortes(D,At):
    #obtener los nombres de los atributos
    atributos = D.get('Atributos', next((k for k, v in D.items() if v == 'Atributos'))
    #indice del atributo
    idx=atributos.index(At)
    #extraer valores y clasificaciones para el atributo
    ejemplos=[(ej[idx],D[ej]) for ej in D if isinstance(ej, tuple) and D[ej] in atributos]
    #ordenar por valores del atributo
    ejemplos.sort(key=lambda x:x[0])
    #calcular puntos de corte
    puntos_corte=[]
    for (v1, c1), (v2, c2) in zip(ejemplos, ejemplos[1:]):
        if c1!=c2: #si las clasificaciones cambian
            puntos_corte.append((v1+v2)/2)
    return puntos_corte
```

Ejemplos:

```
In [26]: cortes(D1,'Temperatura')
# Resultado:
# [19.5, 25.0, 27.5]
```

Out[26]: [19.5, 25.0, 27.5]

```
In [29]: cortes(D1,'Viento')
# Resultado:
# [13.05]
```

Out[29]: [13.05]

```
In [31]: cortes(D2,'AT1')
# Resultado:
# [1.75, 3.8]
```

Out[31]: [1.75, 3.8]

```
In [33]: cortes(D2,'AT3')
# Resultado:
# [5.5]
```

Out[33]: [5.5]

```
In [35]: cortes(D3,'K1')
# Resultado:
# [2.5]
```

Out[35]: [2.5]



27/12/24, 12:20

Práctico G3 Segundo Parcial

## Ejercicio 3 [1 pto]:

Define la función `nuevos_atributos(D)` que reciba como entrada un conjunto de entrenamiento y devuelva una lista de atributos. Esta nueva tupla se construye a partir de la tupla de atributos de `D`, donde cada atributo numérico 'At' se ha sustituido por los atributos 'At<c1', 'At<c2',... 'At<cn', donde `c1,...,cn` son los puntos de corte de `At`.

```
In [38]: # Solución:
def nuevos_atributos(D):
    # Obtener los nombres de los atributos
    atributos = D.get('Atributos', next((k for k, v in D.items() if v == 'Atributos'))
    # Identificar atributos numéricos
    atributos_numericos = at_num(D)
    nuevos_atributos = []

    for atributo in atributos:
        if atributo in atributos_numericos:
            # generar nuevos atributos para cada punto de corte
            puntos_corte = cortes(D, atributo)
            nuevos_atributos.extend([f"{atributo}<{punto}" for punto in puntos_corte])
        else:
            nuevos_atributos.append(atributo)
    return nuevos_atributos
```

```
In [40]: nuevos_atributos(D1)
# Resultado:
# ['Cielo',
#  'Temperatura<19.5',
#  'Temperatura<25.0',
#  'Temperatura<27.5',
#  'Humedad',
#  'Viento<13.05']
```

```
Out[40]: ['Cielo',
          'Temperatura<19.5',
          'Temperatura<25.0',
          'Temperatura<27.5',
          'Humedad',
          'Viento<13.05']
```

```
In [42]: nuevos_atributos(D2)
# Resultado: ['AT1<1.75', 'AT1<3.8', 'AT2', 'AT3<5.5']
```

```
Out[42]: ['AT1<1.75', 'AT1<3.8', 'AT2', 'AT3<5.5']
```

```
In [44]: nuevos_atributos(D3)
# Resultado: ['K1<2.5', 'K2']
```

```
Out[44]: ['K1<2.5', 'K2']
```

## Ejercicio 4 [1 pto]:

Define la función `nuevo_ce(D)` que tome como entrada un conjunto de entrenamiento `D` y devuelva un nuevo conjunto de entrenamiento donde los atributos han sido generados por la función `nuevos_atributos` y donde los valores numéricos han sido sustituidos por



Escanéame

los valores 'SI' o 'NO' según el valor original sea menor o no al punto de corte correspondiente.

```
In [19]: # Solución:
def nuevo_ce(D):
    #obtener nuevos atributos
    nuevos_atr=nuevos_atributos(D)
    #obtener nombre de los atributos originales
    atributos=D.get('Atributos', next((k for k,v in D.items() if v=='Atributos'))
    #inicializar el nuevo conjunto de entrenamiento
    nuevo_D={tuple(nuevos_atr): 'Atributos'}
    #identificar atributos numeros
    atributos_numericos=at_num(D)
    # Crear el nuevo conjunto de ejemplos
    for ej, clasificacion in D.items():
        if isinstance(ej, tuple) and clasificacion in (0, 1):
            nuevo_ej = []
            for i, atributo in enumerate(atributos):
                if atributo in atributos_numericos:
                    # Obtener puntos de corte
                    puntos_corte = cortes(D, atributo)
                    # Transformar valores numéricos en 'SI' o 'NO'
                    nuevo_ej.extend(['SI' if ej[i] < punto else 'NO' for punto in puntos_corte])
                else:
                    nuevo_ej.append(ej[i])
            nuevo_D[tuple(nuevo_ej)] = clasificacion

    return nuevo_D
```

```
In [20]: nuevo_ce(D1)
# Resultado:
# {'Cielo',
#  'Temperatura<19.5',
#  'Temperatura<25.0',
#  'Temperatura<27.5',
#  'Humedad',
#  'Viento<13.05'): 'Atributos',
# ('Soleado', 'SI', 'SI', 'SI', 'NO', 'SI'): 0,
# ('Soleado', 'NO', 'NO', 'SI', 'NO', 'SI'): 0,
# ('Nublado', 'SI', 'SI', 'SI', 'NO', 'SI'): 0,
# ('Lluvia', 'NO', 'SI', 'SI', 'NO', 'NO'): 1,
# ('Lluvia', 'NO', 'SI', 'SI', 'SI', 'NO'): 1,
# ('Nublado', 'NO', 'NO', 'SI', 'SI', 'SI'): 0,
# ('Soleado', 'NO', 'NO', 'NO', 'NO', 'NO'): 1,
# ('Soleado', 'NO', 'SI', 'SI', 'SI', 'NO'): 1,
# ('Soleado', 'NO', 'NO', 'NO', 'SI', 'NO'): 1,
# ('Nublado', 'NO', 'SI', 'SI', 'NO', 'NO'): 1,
# ('Nublado', 'NO', 'NO', 'NO', 'SI', 'NO'): 1,
# ('Lluvia', 'SI', 'SI', 'SI', 'NO', 'SI'): 0}
```

# INTERCAMBIA y AHORRA en LA VUELTA A LA UNI



[webuy.com](https://www.webuy.com)

```
Out[20]: {( 'Cielo',
  'Temperatura<19.5',
  'Temperatura<25.0',
  'Temperatura<27.5',
  'Humedad',
  'Viento<13.05'): 'Atributos',
  ('Soleado', 'SI', 'SI', 'SI', 'NO', 'SI'): 0,
  ('Soleado', 'NO', 'NO', 'SI', 'NO', 'SI'): 0,
  ('Nublado', 'SI', 'SI', 'SI', 'NO', 'SI'): 0,
  ('Lluvia', 'NO', 'SI', 'SI', 'NO', 'NO'): 1,
  ('Lluvia', 'NO', 'SI', 'SI', 'SI', 'NO'): 1,
  ('Nublado', 'NO', 'NO', 'SI', 'SI', 'SI'): 0,
  ('Soleado', 'NO', 'NO', 'NO', 'NO', 'NO'): 1,
  ('Soleado', 'NO', 'SI', 'SI', 'SI', 'NO'): 1,
  ('Soleado', 'NO', 'NO', 'NO', 'SI', 'NO'): 1,
  ('Nublado', 'NO', 'SI', 'SI', 'NO', 'NO'): 1,
  ('Nublado', 'NO', 'NO', 'NO', 'SI', 'NO'): 1,
  ('Lluvia', 'SI', 'SI', 'SI', 'NO', 'SI'): 0}
```

```
In [21]: nuevo_ce(D2)
# Resultado:
# {( 'SI', 'SI', 'B1', 'SI'): 1,
#   ('NO', 'SI', 'B1', 'NO'): 0,
#   ('SI', 'SI', 'B2', 'SI'): 1,
#   ('SI', 'SI', 'B3', 'SI'): 1,
#   ('AT1<1.75', 'AT1<3.8', 'AT2', 'AT3<5.5'): 'Atributos',
#   ('NO', 'SI', 'B3', 'NO'): 0,
#   ('NO', 'NO', 'B3', 'SI'): 1}
```

```
Out[21]: {( 'SI', 'SI', 'B1', 'SI'): 1,
  ('NO', 'SI', 'B1', 'NO'): 0,
  ('SI', 'SI', 'B2', 'SI'): 1,
  ('SI', 'SI', 'B3', 'SI'): 1,
  ('AT1<1.75', 'AT1<3.8', 'AT2', 'AT3<5.5'): 'Atributos',
  ('NO', 'SI', 'B3', 'NO'): 0,
  ('NO', 'NO', 'B3', 'SI'): 1}
```

```
In [22]: nuevo_ce(D3)
# Resultado:
# {( 'SI', 'T1'): 0,
#   ('SI', 'T2'): 0,
#   ('NO', 'T3'): 1,
#   ('NO', 'T4'): 1,
#   ('K1<2.5', 'K2'): 'Atributos'}
```

```
Out[22]: {( 'SI', 'T1'): 0,
  ('SI', 'T2'): 0,
  ('NO', 'T3'): 1,
  ('NO', 'T4'): 1,
  ('K1<2.5', 'K2'): 'Atributos'}
```